

# Poker Tab: Requirement Analysis Document

GitHub: <https://github.com/HiddenOffset/PokerTab-Group15.git>

Poker Tab is a locally run C++ desktop application that tracks buy-ins, re-buys and cash-outs for a casual poker session and computes the fewest transactions that settle everyone up. Team: Travis Trinidad (Narrative Lead), Nishanth Mahendran (Front-End), Matthew Rohrer (Back-End), David Martindale (Problem Framer).

## Contribution Table

One row per team member. Section assignments below follow each member's proposal role; adjust to match who actually writes what. Every member must author at least one RAD section and have at least one merged pull request; the Part 0 README pull request does not count. Contribution points are graded per row.

| Team Member        | Role           | RAD Sections Authored                                                                      | REQ Supported by Your Code                     | Your Merged Pull Request (link) | Pull Request You Reviewed (link) |
|--------------------|----------------|--------------------------------------------------------------------------------------------|------------------------------------------------|---------------------------------|----------------------------------|
| Travis Trinidad    | Narrative Lead | 2.1 Product Perspective, 2.2 Product Functions, 2.3 User Characteristics                   | REQ-1, REQ-2, REQ-3 (session create/save/load) |                                 |                                  |
| Nishanth Mahendran | Front-End      | 3.6 Corrections and Undo, 3.7 Dashboard, NFR-7 and NFR-8 (usability)                       | REQ-23, REQ-24, REQ-26, REQ-27, REQ-28         |                                 |                                  |
| Matthew Rohrer     | Back-End       | 3.1 through 3.5 (Session, Players, Buy-ins, Cash-outs, Settlement), Appendix A, Appendix B | REQ-9, REQ-16, REQ-17, REQ-18, REQ-19, REQ-22  |                                 |                                  |

| Team Member      | Role           | RAD Sections Authored                                                                                                           | REQ Supported by Your Code                             | Your Merged Pull Request (link) | Pull Request You Reviewed (link) |
|------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------------------------|----------------------------------|
| David Martindale | Problem Framer | 1. Introduction (1.1 through 1.4), 2.4 Constraints, 4. Non-Functional Requirements, 5. Assumptions and Dependencies, Appendix C | REQ-5, REQ-6, REQ-11 (player add and input validation) |                                 |                                  |

## 1. Introduction

### 1.1 Purpose

This document defines the requirements for Poker Tab. It establishes a shared understanding among the four team members, the course instructor, and end users of what the system must do. It is the basis for design, implementation, and testing; every functional requirement below has an acceptance criterion a tester can check.

### 1.2 Scope

The system will provide a single-user desktop application that acts as the accountant for one poker session: it records players, money in, money out, and produces a minimal settlement plan at the end of the night.

In scope:

- Managing one session at a time: players, buy-ins, re-buys, cash-outs
- Computing each player's net result and a minimized list of who pays whom
- Saving and reloading a session from local disk
- Running offline as a downloadable executable

Out of scope:

- Dealing cards, tracking hands, pots, blinds, or any game play
- Online play, multi-device sync, or a server component
- Processing real payments (Venmo, Zelle, etc.)
- User accounts, login, or cloud storage

### 1.3 Definitions, Acronyms, and Abbreviations

| Term                  | Meaning                                                                            |
|-----------------------|------------------------------------------------------------------------------------|
| RAD                   | Requirement Analysis Document                                                      |
| UI                    | User Interface                                                                     |
| Accountant            | The one person operating Poker Tab during a session, usually the host              |
| Session               | One night of play from first buy-in to final settlement                            |
| Buy-in                | Cash a player puts into the game to receive chips                                  |
| Re-buy                | Any buy-in after a player's first one                                              |
| Cash-out              | Dollar value of a player's chips when they leave or the game ends                  |
| Net                   | Cash-out minus total buy-ins for one player (positive = won)                       |
| Settlement            | The list of payments that zeros every player's net                                 |
| MUST / SHOULD / COULD | MoSCoW priority: required for the deliverable / expected if time allows / optional |

### 1.4 References

- Poker Night Manager project proposal (team, Fall 2026)
- Course-provided RAD template
- IEEE 830-1998, Recommended Practice for Software Requirements Specifications (structure and MoSCoW-style prioritization)
- Debt-simplification (minimum cash flow) algorithm, the standard greedy approach used by expense-splitting apps such as Splitwise

## 2. Overall Description

### 2.1 Product Perspective

Poker Tab is a standalone desktop application with no server, network, or external service dependencies. It replaces the pen-and-paper or phone-notes ledger a host keeps during a game. Session data lives in a single file on the local disk of the machine running the program.

flowchart LR

```
A[Accountant] -->|adds players, buy-ins, cash-outs| U[UI layer]
U --> L[Session logic - C++]
```

```
L --> S[Settlement calculator]
L <--> F[Local session file]
S -->|who pays whom| U
```

The accountant is the only user. All input flows through one dashboard into the C++ session logic, which owns the data, persists it locally, and calls the settlement calculator on demand.

## 2.2 Product Functions (High-Level)

- Create, save, and reload a poker session
- Add and remove players
- Record buy-ins and re-buys per player, with a running total
- Record each player's cash-out amount
- Display each player's current balance (total in, cash-out, net)
- Validate that total cash-outs equal total buy-ins before settling
- Compute and display a settlement plan with the minimum number of payments
- Allow correction of any mis-entered transaction

## 2.3 User Characteristics

- Primary user: the session accountant, typically the host of a casual college poker game
- Secondary users: the other players, who look at the screen at the end of the night but do not operate the program
- Technical expertise: basic computer literacy; no training beyond reading button labels
- Access environment: a laptop at the table, running the downloaded executable, possibly with no internet
- Context of use: the accountant is also playing, so entries happen in seconds between hands and mistakes must be easy to undo

## 2.4 Constraints

- Business: the project must be completed within the course term by a four-person team and follow the course AI-use disclosure policy
- Technical: core logic written in C++17 or later; the build must produce a single downloadable executable; the app must run fully offline and store data in local files
- Technical: UI toolkit is undecided; candidates are Qt (mature, cross-platform, heavier to learn), Dear ImGui (lightweight, immediate-mode, easy to embed), and wxWidgets (native look, moderate learning curve). Decision due before front-end work begins.
- Platform: Windows 10+ is the required target; macOS and Linux are desired but not required
- Process: all code committed to the shared GitHub repository via VS Code; each feature released as a GitHub Release after testing
- Regulatory: no personal data beyond first names and dollar amounts is stored; data never leaves the device, so no privacy-law obligations apply beyond not exposing the file

## 3. Functional Requirements

Each requirement is numbered, prioritized (MUST / SHOULD / COULD), and paired with an acceptance criterion a tester can verify without reading the code. Requirements are grouped by feature area; numbering is continuous.

### 3.1 Session Management

| ID    | Requirement                                                                                        | Priority | Acceptance criterion                                                                                                               |
|-------|----------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------------|
| REQ-1 | The system shall create a new, empty poker session with a user-supplied name and the current date. | MUST     | After clicking New Session and entering a name, the dashboard shows that name, today's date, zero players, and a \$0.00 total pot. |
| REQ-2 | The system shall save the current session to a local file.                                         | MUST     | After Save, a file exists on disk; reopening it restores every player, transaction, and cash-out exactly.                          |
| REQ-3 | The system shall load a previously saved session from a local file.                                | MUST     | Loading a saved file reproduces the same player list, balances, and total pot shown before saving.                                 |
| REQ-4 | The system shall auto-save after every recorded transaction.                                       | SHOULD   | Killing the process immediately after a buy-in and reloading shows that buy-in present.                                            |

### 3.2 Player Management

| ID    | Requirement                                                       | Priority | Acceptance criterion                                                                                  |
|-------|-------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------|
| REQ-5 | The system shall add a player to the session by name.             | MUST     | Entering a name and clicking Add Player shows the player on the roster with \$0.00 in and \$0.00 out. |
| REQ-6 | The system shall reject a duplicate player name within a session. | MUST     | Adding a name already on the roster shows an error and leaves the roster unchanged.                   |
| REQ-7 | The system shall remove a player who                              | SHOULD   | Remove is enabled only for players with zero buy-ins; clicking it                                     |

| ID    | Requirement                                                  | Priority | Acceptance criterion                                                                                 |
|-------|--------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------|
|       | has no recorded transactions.                                |          | drops the player from the roster.                                                                    |
| REQ-8 | The system shall support at least 10 players in one session. | MUST     | Ten players can be added and all appear on the dashboard without scrolling off-screen or truncation. |

### 3.3 Buy-ins and Re-buys

| ID     | Requirement                                                                                                | Priority | Acceptance criterion                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------|
| REQ-9  | The system shall record a buy-in of a positive dollar amount for a selected player.                        | MUST     | After entering \$20.00 for a player, their total-in reads \$20.00 and the total pot increases by \$20.00. |
| REQ-10 | The system shall record any number of additional buy-ins (re-buys) for a player and sum them.              | MUST     | Three buy-ins of \$20, \$20, \$10 for one player show total-in \$50.00 and a re-buy count of 2.           |
| REQ-11 | The system shall reject a buy-in that is zero, negative, non-numeric, or has more than two decimal places. | MUST     | Entering -5, 0, abc, or 10.001 shows an error and records nothing.                                        |
| REQ-12 | The system shall offer a configurable default buy-in amount to reduce typing.                              | SHOULD   | Setting the default to \$20 pre-fills the buy-in field with 20.00 for every player.                       |
| REQ-13 | The system shall keep a timestamped transaction log of every buy-in per player.                            | SHOULD   | Selecting a player shows a list of their buy-ins with time and amount, in order.                          |

### 3.4 Cash-outs

| ID     | Requirement                                                                                                    | Priority | Acceptance criterion                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------|
| REQ-14 | The system shall record a cash-out amount (chip value, zero or greater) for a selected player.                 | MUST     | Entering \$35.00 shows the player's cash-out as \$35.00 and marks them cashed out.                                            |
| REQ-15 | The system shall compute and display each player's net as cash-out minus total buy-ins.                        | MUST     | A player with \$50 in and \$35 out shows net -\$15.00; one with \$20 in and \$60 out shows +\$40.00.                          |
| REQ-16 | The system shall verify that total cash-outs equal total buy-ins before settlement and report the discrepancy. | MUST     | If buy-ins total \$200 and cash-outs total \$195, the Settle button is disabled and a message reads "\$5.00 unaccounted for". |

### 3.5 Settlement

| ID     | Requirement                                                                                                 | Priority | Acceptance criterion                                                                        |
|--------|-------------------------------------------------------------------------------------------------------------|----------|---------------------------------------------------------------------------------------------|
| REQ-17 | The system shall generate a list of payments (payer, payee, amount) that brings every player's net to zero. | MUST     | For any balanced session, applying every listed payment leaves all nets at \$0.00.          |
| REQ-18 | The system shall minimize the number of payments in the settlement.                                         | MUST     | For the test cases in Appendix A, the number of payments produced equals the known minimum. |
| REQ-19 | The system shall display the settlement on the dashboard with payer, payee, and amount clearly labeled.     | MUST     | Each line reads as "Alice pays Bob \$15.00"; amounts owed and received are color-coded.     |

| ID     | Requirement                                                  | Priority              | Acceptance criterion                                                                |
|--------|--------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------|
| REQ-20 | The system shall export the settlement as a plain-text file. | SHOULD → <u>COULD</u> | Clicking Export writes a .txt file containing every player's net and every payment. |

### 3.6 Corrections and Undo

| ID     | Requirement                                                                                    | Priority | Acceptance criterion                                                                                                 |
|--------|------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------|
| REQ-21 | The system shall show a confirmation message after every recorded transaction.                 | MUST     | After each buy-in or cash-out, a message such as "Recorded \$20.00 buy-in for Alice" appears for at least 3 seconds. |
| REQ-22 | The system shall allow the most recent transaction to be undone.                               | MUST     | Clicking Undo after a \$20 buy-in removes it; the player's total-in and the pot return to their prior values.        |
| REQ-23 | The system shall allow any individual transaction to be edited or deleted from a player's log. | SHOULD   | Editing a \$20 entry to \$25 in the log updates the player's total-in and the pot by \$5.00.                         |

### 3.7 Dashboard

| ID     | Requirement                                                                                                        | Priority | Acceptance criterion                                                                                           |
|--------|--------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------|
| REQ-24 | The system shall present all primary actions (add player, buy-in, cash-out, settle, undo) on a single main screen. | MUST     | Every action in the list is reachable with one click from the dashboard, with no navigation to another window. |
| REQ-25 | The system shall display, for every player, name, number of buy-ins, total in,                                     | MUST     | All five columns are visible for every player at the default window size.                                      |

| ID     | Requirement                                                               | Priority              | Acceptance criterion                                                                             |
|--------|---------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|
|        | cash-out, and net on the dashboard.                                       |                       |                                                                                                  |
| REQ-26 | The system shall display the total pot (sum of all buy-ins) at all times. | <b>MUST</b>           | The pot figure updates within one second of any buy-in, undo, or edit.                           |
| REQ-27 | The system shall use color to distinguish positive from negative net.     | SHOULD → <u>COULD</u> | Positive nets render in green, negative in red, zero in neutral; a colorblind-safe pair is used. |

#### 4. Non-Functional Requirements

The template's defaults (concurrent users, uptime, encryption) fit a web service, not an offline single-user desktop tool, so they are replaced below with measurable targets that match Poker Tab.

| ID    | Category    | Requirement                                                                                             | Verification                                               |
|-------|-------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| NFR-1 | Performance | Any transaction (add, buy-in, cash-out, undo) shall update the dashboard within 1 second.               | Stopwatch test with 10 players and 50 transactions loaded. |
| NFR-3 | Performance | The executable shall launch to a usable dashboard within 3 seconds on a 2020-or-newer laptop.           | Manual timing on each team member's machine.               |
| NFR-6 | Reliability | Every functional <b>MUST</b> requirement shall have at least one automated unit test in the repository. | CI run shows a test per REQ-ID.                            |
| NFR-8 | Usability   | All dollar amounts shall display with two decimal places and a dollar sign.                             | Visual inspection of dashboard and export.                 |
| NFR-9 | Portability | The application shall run on Windows 10                                                                 | Release tested on Windows; other                           |

| ID     | Category        | Requirement                                                                                                                         | Verification                                                 |
|--------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
|        |                 | and later; <del>macOS and Linux builds are desired.</del>                                                                           | platforms noted in release notes.                            |
| NFR-10 | Portability     | The application shall run with no internet connection and no installer beyond unzipping.                                            | Run with network disabled on a clean machine.                |
| NFR-11 | Security        | Session files shall contain only player first names and dollar amounts; no other personal data.                                     | Inspect the saved file format.                               |
| NFR-12 | Maintainability | Session logic and settlement shall live in a UI-independent C++ library so the UI toolkit can change without rewriting them.        | Build the core as a static library with its own test target. |
| NFR-13 | Maintainability | Each GitHub Release shall be tagged with a version number and include release notes listing the features and bug fixes it contains. | Inspect the Releases page.                                   |

## 5. Assumptions and Dependencies

### Assumptions:

- One person (the accountant) operates the program; players trust that person's entries
- All players buy in and cash out in the same currency (USD) and in whole cents
- Chip values are converted to dollars by the accountant before entry; the app does not track chip denominations
- The accountant has a laptop at the table; no internet access is assumed
- Total cash-outs equal total buy-ins in a correctly counted game; any difference is a counting error, not a feature
- Player names are first names or nicknames; no two players share one within a session

### Dependencies:

- A C++17-capable compiler on each developer machine (MSVC, GCC, or Clang) and CMake for a reproducible build
- The UI toolkit chosen in Section 2.4 (Qt, Dear ImGui, or wxWidgets), which must be either statically linked or bundled with the executable
- A unit-test framework (Catch2 or GoogleTest) for the settlement and session-logic tests
- A serialization approach for the session file (a JSON library such as nlohmann/json, or a hand-written CSV format)
- GitHub for source control, issue tracking, and Releases; VS Code as the shared editor
- Availability of real poker sessions for the end-to-end testing described in the proposal

## 6. Appendices

### Appendix A: Settlement Algorithm and Test Cases

The settlement uses the standard debt-simplification greedy algorithm. Compute each player's net in integer cents; split players into creditors (net > 0) and debtors (net < 0); repeatedly match the largest debtor to the largest creditor, transfer the smaller of the two magnitudes, and remove whoever reaches zero. This yields at most  $n - 1$  payments for  $n$  players with nonzero net and is optimal for the small groups Poker Tab targets.

flowchart TD

```

A[Compute net per player in cents] --> B{All nets sum to 0?}
B -->|No| C[Report discrepancy, block settle]
B -->|Yes| D[Sort debtors and creditors by magnitude]
D --> E[Match largest debtor to largest creditor]
E --> F[Pay min of the two amounts]
F --> G[Remove any player now at 0]
G --> H{Anyone left?}
H -->|Yes| E
H -->|No| I[Return payment list]

```

REQ-19's acceptance tests. Each case lists nets after cash-out; the minimum payment count is the target the implementation must hit.

| Case            | Nets (cents)                                    | Minimum payments | Expected settlement                                 |
|-----------------|-------------------------------------------------|------------------|-----------------------------------------------------|
| A1: two players | Alice +2000, Bob -2000                          | 1                | Bob pays Alice \$20.00                              |
| A2: one winner  | Alice +3000, Bob -1000, Carol -2000             | 2                | Bob pays Alice \$10.00;<br>Carol pays Alice \$20.00 |
| A3: chain       | Alice +1500, Bob +500, Carol -2000              | 2                | Carol pays Alice \$15.00;<br>Carol pays Bob \$5.00  |
| A4: exact pairs | Alice +1000, Bob -1000, Carol +2500, Dave -2500 | 2                | Bob pays Alice \$10.00;<br>Dave pays Carol \$25.00  |
| A5: cents       | Alice +1010, Bob -505, Carol -505               | 2                | Bob pays Alice \$5.05;<br>Carol pays Alice \$5.05   |

| Case                   | Nets (cents)                            | Minimum payments | Expected settlement                      |
|------------------------|-----------------------------------------|------------------|------------------------------------------|
| A6: break-even players | Alice +4000, Bob 0, Carol 0, Dave -4000 | 1                | Dave pays Alice \$40.00                  |
| A7: unbalanced         | Alice +2000, Bob -1500                  | 0                | Refused: \$5.00 unaccounted for (REQ-17) |

## Appendix B: Data Model

Money is stored as a 64-bit integer count of cents (NFR-5). The session file serializes these three records.

| Entity      | Fields                                                                 | Notes                                              |
|-------------|------------------------------------------------------------------------|----------------------------------------------------|
| Session     | id, name, date, default_buy_in_cents, players[]                        | One file per session                               |
| Player      | id, name, transactions[], cash_out_cents (optional), cashed_out (bool) | net = cash_out - sum(transactions)                 |
| Transaction | id, timestamp, amount_cents, kind (buy_in)                             | Only buy-ins; cash-out is a single field on Player |

## Appendix C: Open Decisions

- ☐ UI toolkit: Qt, Dear ImGui, or wxWidgets (Section 2.4). Recommendation: Dear ImGui if the priority is shipping a working executable quickly; Qt if a native-looking product and resume value matter more.
- ☐ Session file format: JSON via nlohmann/json (readable, one header file) or hand-rolled CSV (no dependency, more parsing code).
- ☐ Test framework: Catch2 (single header, simplest to add) or GoogleTest.
- ☐ Target platforms beyond Windows: decide whether macOS/Linux builds are in the first release or deferred.
- ☐ Proposal inconsistency to resolve: the Solution Framer and Front-End sections describe a desktop executable, but the Back-End section refers to a mobile app. This RAD assumes desktop only.

## Appendix D: Glossary of UI Terms

| Term   | Meaning on the dashboard                                       |
|--------|----------------------------------------------------------------|
| Roster | The table listing every player and their five figures (REQ-27) |
| Pot    | Running sum of all buy-ins in the session (REQ-28)             |

| Term   | Meaning on the dashboard                                                         |
|--------|----------------------------------------------------------------------------------|
| Settle | The action that validates totals (REQ-17) and produces the payment list (REQ-18) |
| Undo   | Reverses the single most recent transaction (REQ-24)                             |